



# AUTOENCODERS

## Redes Neurais Não Supervisionadas

Deep Learning | MBA Data Science | FIAP

Profa. Juliana Scudilio

# AGENDA

## 01 **Aprendizado Não Supervisionado**

Conceito e diferenças

## 02 **O que são Autoencoders?**

Arquitetura e intuição

## 03 **Encoder, Latent Space, Decoder**

As 3 partes fundamentais

## 04 **Função de Perda**

Erro de reconstrução (MSE)

## 05 **Aplicações Reais**

Fraude, anomalias, compressão

## 06 **Tipos de Autoencoders**

Vanilla, Denoising, VAE, Sparse

## 07 **Hands-on Python**

Implementação com TensorFlow/Keras



# APRENDIZADO NÃO SUPERVISIONADO

*Modelos que aprendem padrões sem rótulos (labels)*

## SUPERVISIONADO

- Dados com rótulos ( $X, y$ )
- Objetivo: prever  $y$  dado  $X$
- Ex: Classificação, Regressão
- Métricas: accuracy, F1, RMSE

## NÃO SUPERVISIONADO

- Dados sem rótulos (apenas  $X$ )
- Objetivo: descobrir estrutura
- Ex: Clustering, Autoencoders
- Métricas: silhouette, erro recon.

# O QUE SÃO AUTOENCODERS?

Uma rede neural que aprende a comprimir e reconstruir dados



## Não Supervisionado

Não precisa de labels.  
Aprende padrões sozinho.



## Data-Specific

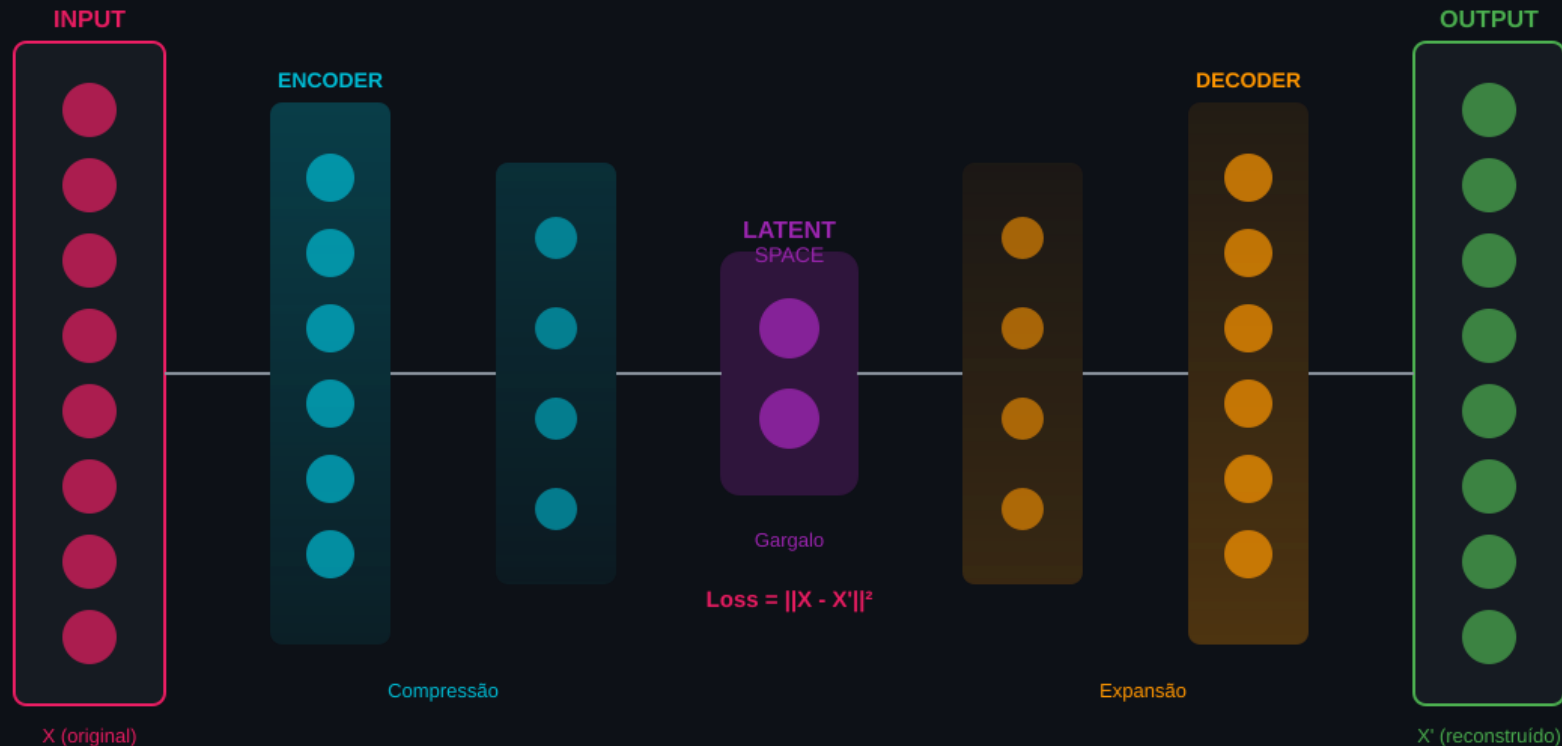
Só comprime dados  
similares ao treinamento.



## Lossy

A reconstrução é  
aproximada, não exata.

# ARQUITETURA DO AUTOENCODER



*Input → Encoder → Latent Space → Decoder → Output  $\approx$  Input*

# ANALOGIA: ZIP INTELIGENTE



Se a reconstrução é boa → o modelo aprendeu a essência dos dados



Diferença do ZIP real: o autoencoder é lossy (com perda). Ele não recupera 100% dos dados, mas aprende a representação mais relevante.

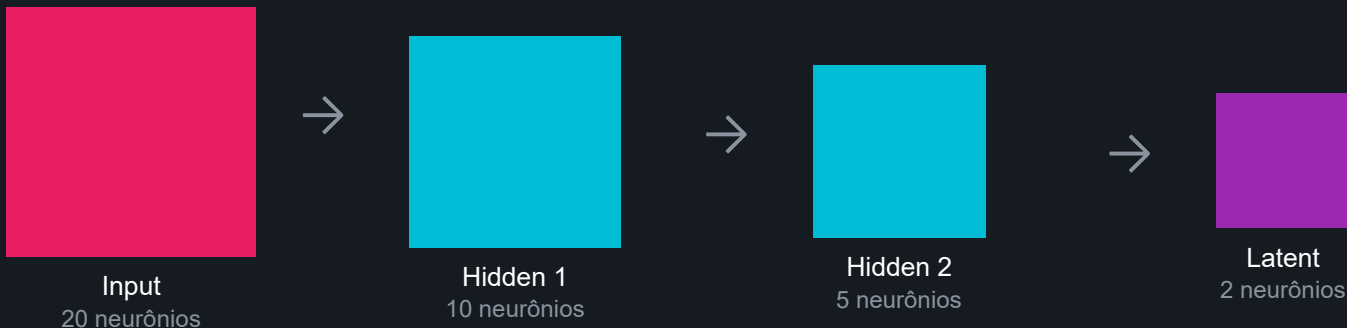
# ENCODER: COMPRESSÃO DOS DADOS



## Função do Encoder

Reduz progressivamente a dimensionalidade dos dados de entrada, forçando a rede a aprender apenas as características mais importantes.

### Exemplo de Encoder (tabular data)



# LATENT SPACE & DECODER



## Latent Space (Gargalo)

- É o ponto de menor dimensão
- Contém a representação comprimida
- Aqui está o "conhecimento" aprendido
- Quanto menor, mais compressão
- **Usado para Feature Extraction!**



## Decoder (Reconstrução)

- Processo inverso do Encoder
- Expande de volta à dimensão original
- Saída deve ser  $\approx$  Entrada
- Erro de reconstrução mede qualidade
- **$MSE = \Sigma(X - X')^2 / n$**



# FUNÇÃO DE PERDA

*O autoencoder minimiza o erro de reconstrução entre entrada e saída*

## Mean Squared Error (MSE)

$$L(X, X') = (1/n) \times \sum (X_i - X'_i)^2$$

X = entrada original | X' = saída reconstruída | n = número de features



### Loss Baixo

O modelo aprendeu bem a comprimir e reconstruir. Os dados passam pelo gargalo sem perder informação essencial.



### Loss Alto

Os dados não foram bem reconstruídos. Pode indicar: dados atípicos (anomalia!) ou modelo precisa de mais treinamento.

# APLICAÇÕES REAIS



## Detecção de Fraude

Treina com dados normais. Erro alto = anomalia (possível fraude)



## Manutenção Preditiva

Monitora sensores. Desvio na reconstrução = falha iminente



## Compressão de Dados

Reduz dimensão de imagens, vídeos e big data.  
Reconstrução de audios, videos etc



## Redução de Dimensionalidade

Alternativa não-linear ao PCA. Captura relações complexas



## Feature Extraction

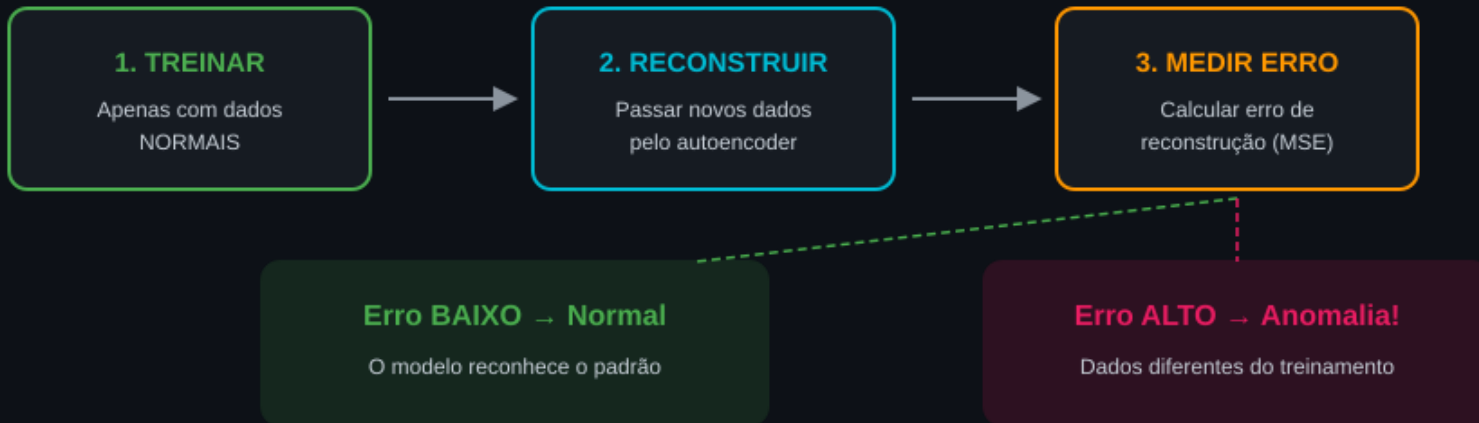
Latent space como input para outros modelos de ML



## Cybersegurança

Detecção de intrusões e tráfego anômalo em redes

# DETECÇÃO DE ANOMALIAS COM AUTOENCODERS



Insight MBA: Em cenários reais de fraude, < 1% dos dados são fraudes. Não há labels suficientes para treinar modelos supervisionados. Autoencoders resolvem isso aprendendo o que é "normal".

# TIPOS DE AUTOENCODERS

## Vanilla (Simples)

Arquitetura básica.  
Encoder → Latent → Decoder.  
Base para entender todos os outros.

## Denoising

Recebe dados com ruído.  
Aprende a reconstruir versão limpa.  
Ótimo para limpeza de dados.

## Sparse

Adiciona regularização L1.  
Força ativações esparsas.  
Melhores features comprimidas.

## Variational (VAE)

Latent space probabilístico.  
Gera novos dados (generativo).  
Base para modelos generativos.

# AUTOENCODER vs PCA

*Ambos fazem redução de dimensionalidade, mas de formas diferentes*

| Critério              | PCA                    | Autoencoder             |
|-----------------------|------------------------|-------------------------|
| Tipo de transformação | Linear                 | Não-linear              |
| Complexidade          | Baixa (closed-form)    | Alta (treinamento NN)   |
| Interpretabilidade    | Alta (eigenvalues)     | Baixa (black box)       |
| Relações complexas    | Não captura            | Captura (deep learning) |
| Velocidade            | Rápido                 | Lento                   |
| Dados necessários     | Funciona com poucos    | Precisa de volume alto  |
| Melhor para           | Dados lineares/simples | Dados complexos/imagens |

**"Quando não temos respostas, a inteligência está em descobrir padrões — e é exatamente isso que os autoencoders fazem."**

# HANDS-ON: AUTOENCODER EM PYTHON



```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

# Definir dimensão de entrada
input_dim = 20

# Encoder
input_layer = Input(shape=(input_dim,))
encoded = Dense(10, activation='relu')(input_layer)
encoded = Dense(5, activation='relu')(encoded) # bottleneck

# Decoder
decoded = Dense(10, activation='relu')(encoded)
decoded = Dense(input_dim, activation='sigmoid')(decoded)

# Compilar
autoencoder = Model(input_layer, decoded)
autoencoder.compile(optimizer='adam', loss='mse')
```

# TAKEAWAYS



Autoencoders aprendem a ESTRUTURA dos dados, não classes



Encoder comprime → Latent Space guarda a essência → Decoder reconstrói



Detecção de anomalias: erro de reconstrução alto = dado anômalo



Alternativa não-linear ao PCA para redução de dimensionalidade



Latent space é poderoso para Feature Extraction em pipelines de ML



Cenários ideais: dados sem labels, anomalias raras, alta dimensionalidade